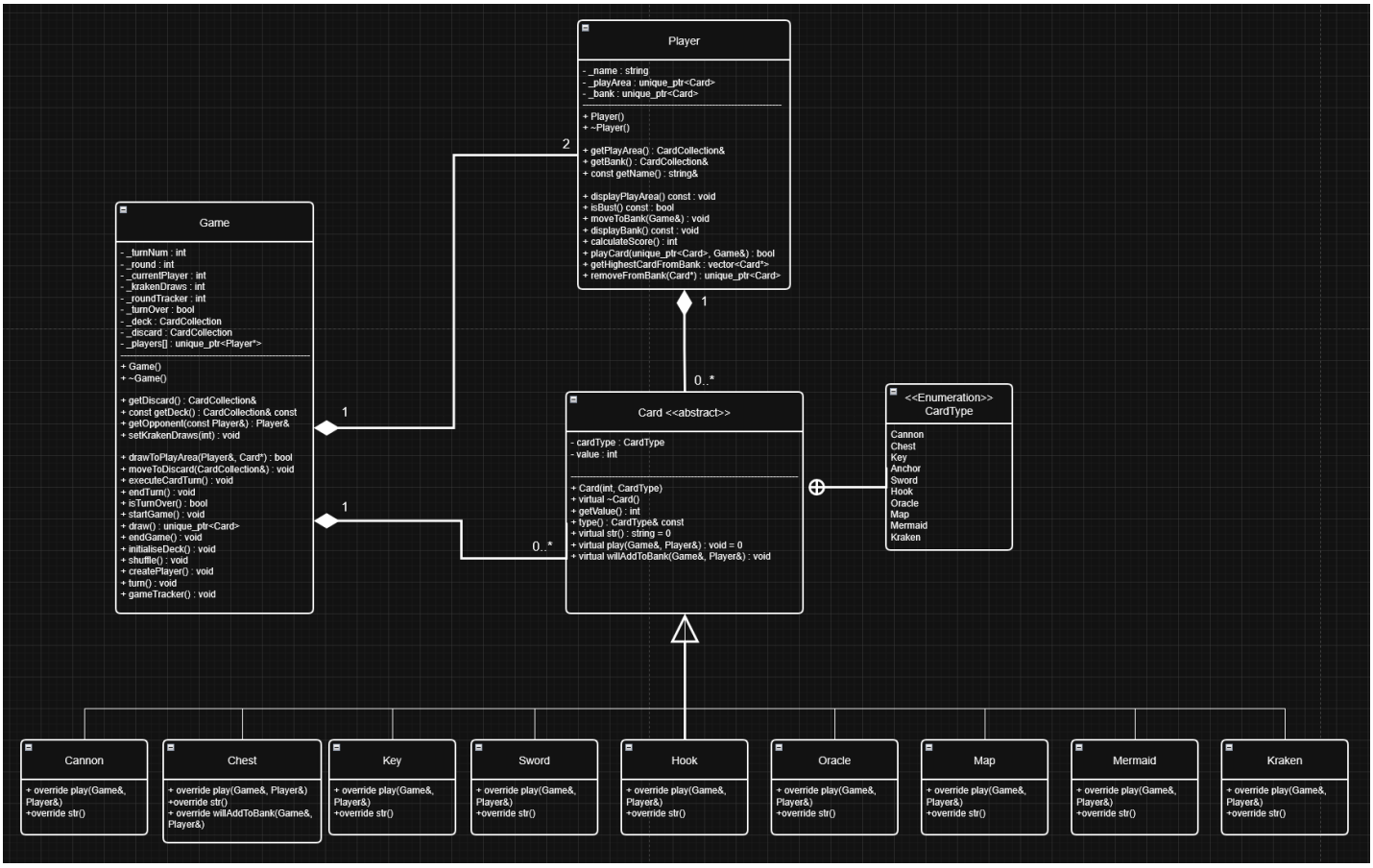
# System Design Document

## Class Diagram




## Design Decisions

### Use of C++ Standard Libraries

- **std::map** was used to map cardTypes (suits) with their values and was used for grouping suits and displaying cards in the bank, tracking the highest cards to calculate the score, and getting the top value cards from an opponent's bank
- **std::set** was implemented to check if the player is going to bust by first adding a card to the player's current play area and then creating a set. If the set ends up smaller than the play area, a suit was duplicated, so the player busts.
- **std::vector** was used to hold the collection of cards in the deck, discard, play area, and bank
- **Arrays** were used by fixing the number of player names to be randomly chosen from and to fix the number of players in the game.

**Polymorphism**

- Each card type is a subclass of the “Card” class, which is an abstract class. This way, each card ability can be written for that specific card, and only run when that card is drawn
- The willAddToBank function in Card is overridden in the Chest class with the Chest/Key ability functionality, so that ability is only run from one class to prevent duplication

**Abstraction**

- Game.cpp handles the game logic, Player handles the player’s bank, score, and play area, and the main file that runs the game (Plunder.cpp) only needs to create a Game object and call game.startGame(). This separation of concerns allows for extension without breaking the rest of the classes. It also makes the code more maintainable.

**Dynamic Allocation**

- Unique pointers were used to reference each created card and player
- References to objects through functions were used throughout the entire project. Objects are passed by reference to avoid copying and to let the abilities modify the game state
- Abilities use non-owning raw Card* pointers from .get(), and ownership stays with the unique pointers
- Ownership is transferred with std::move

**Memory Management**

- Since unique pointers delete when they go out of scope, there wasn’t any need for manual pointer deletion
- Verified leak-free with AddressSanitizer

**Const Declarations**

- Most getter functions were declared const, except for getDiscard(), getBank(), and getPlayArea(), since they were used and modified through card abilities like Map and Chest
- Any method that did not modify any data was declared const, such as displayBank/PlayArea(), and isBust().

**Initialisation**

- Initialiser lists were used in the Game and Card constructor to initialise all member variables

**Game Logic**

- The discard pile acts as a stack, so cards that are discarded will use Last-In-First-Out principles when an ability that sees the discard pile is used (Map for instance)
- For the Kraken ability, added a separate draw loop within the turn function in Game using an int krakenDraws variable to prevent duplicating the same code in the Kraken class

**Code Style**

- All header files implemented #ifndef, #define, and #endif
- All argument names are informative
- PascalCase classes and camelCase methods were used
- Underscore was used for private member variables
- An enum class was used to define the card types

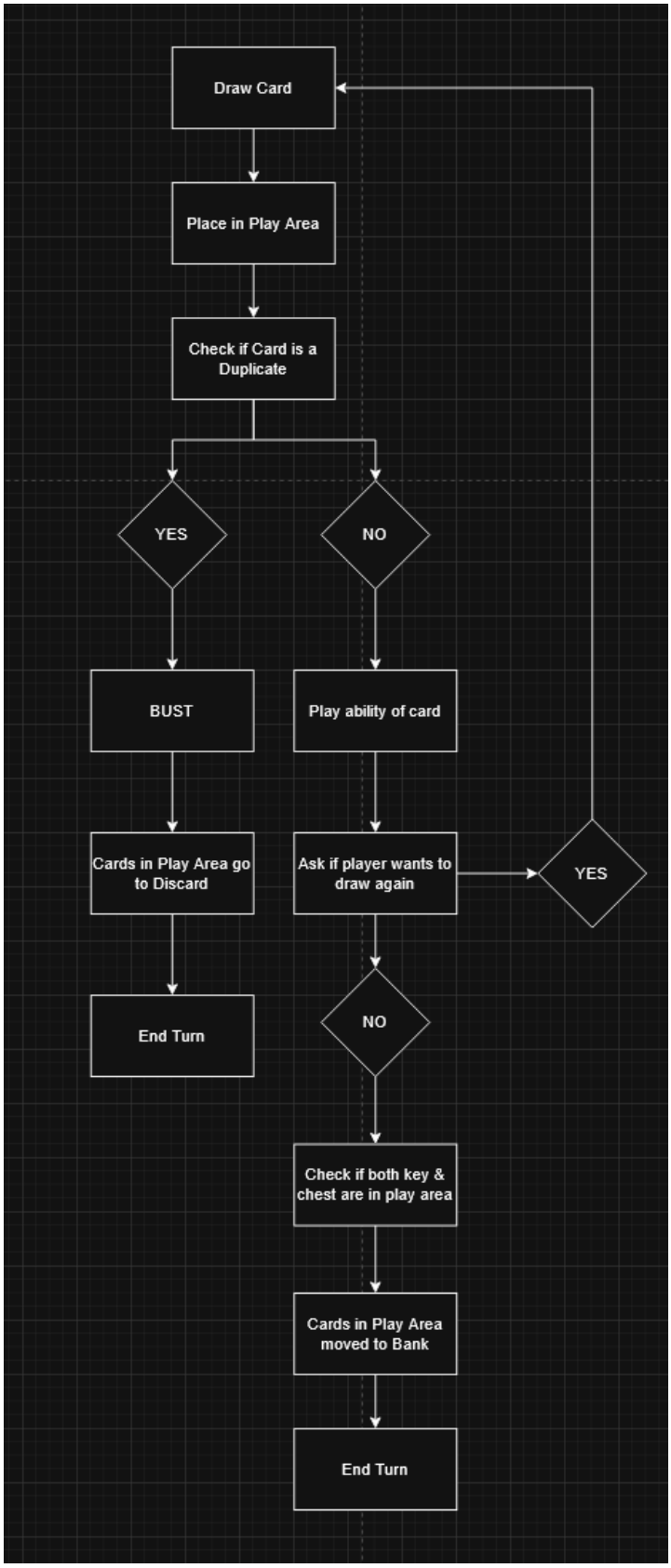
**Error Handling**

- Input handling is implemented so that no crashes/errors occur, and will continually ask the player to input again until it is correct
- Added an input at the very end of the game to prevent the game from quitting automatically, so that you can see the winner of the game

**External Libraries**

- No external libraries were used because this project was a learning exercise in C++

## Flowchart of Gameplay Loop




Flow of the gameplay to provide visualisation to what is occurring, and shows the order of actions